

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 5_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a singly linked list and two integers M and N, implement a program to traverse the linked list such that you retain every M node and then delete the next N nodes.

Continue this process until the end of the linked list.

Input Format

The first line consists of two integers M and N, separated by a space, representing the values to retain and delete.

The second line consists of a series of integers separated by spaces, representing the values of the linked list nodes.

The last input is -1 which will terminate the inputs.

Output Format

The output prints the linked list after applying the process of retaining every M node and deleting the next N nodes represented as a series of integers separated by "->".

Sample Test Case

Input: 2 2

1 2 3 4 5 6 7 8 -1

Output: 1->2->5->6

Answer

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

```
struct Node {
    int data;
    struct Node* next;
};
```

// create node

```
struct Node* create(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// process M retain, N delete

```
void modifyList(struct Node* head, int M, int N) {
    struct Node* curr = head;
```

```
    while (curr != NULL) {
        // retain M nodes
        for (int i = 1; i < M && curr != NULL; i++) {
            curr = curr->next;
        }
```

```
        if (curr == NULL) break;
```

```
        // delete next N nodes
```

```

    struct Node* temp = curr->next;
    for (int i = 0; i < N && temp != NULL; i++) {
        struct Node* del = temp;
        temp = temp->next;
        free(del);
    }

    curr->next = temp;
    curr = temp;
}
}

```

```

// print list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL)
            printf("->");
        temp = temp->next;
    }
}

```

```

int main() {
    int M, N, x;
    scanf("%d %d", &M, &N);

    struct Node *head = NULL, *tail = NULL;

    // input list
    while (1) {
        scanf("%d", &x);
        if (x == -1) break;

        struct Node* newNode = create(x);
        if (head == NULL) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }
}

```

```
    modifyList(head, M, N);  
    printList(head);  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Imagine you are working on a text processing tool and need to implement a feature that allows users to insert characters at a specific position.

Implement a program that takes user inputs to create a singly linked list of characters and inserts a new character after a given index in the list.

Input Format

The first line of input consists of an integer N, representing the number of characters in the linked list.

The second line consists of a sequence of N characters, representing the linked list.

The third line consists of an integer index, representing the index(0-based) after which the new character node needs to be inserted.

The fourth line consists of a character value representing the character to be inserted after the given index.

Output Format

If the provided index is out of bounds (larger than the list size):

1. The first line prints "Invalid index".
2. The second line prints "Updated list: " followed by the unchanged linked list values.

Otherwise, prints "Updated list: " followed by the updated space separated linked

list after inserting the new character after the given index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

a b c d e

2

X

Output: Updated list: a b c X d e

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    char data;  
    struct Node* next;  
};
```

```
// create node
```

```
struct Node* create(char data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
// print list
```

```
void printList(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%c ", temp->data);  
        temp = temp->next;  
    }  
}
```

```
int main() {
```

```
int n, index;
scanf("%d", &n);

struct Node *head = NULL, *tail = NULL;
```

```
// input characters
for (int i = 0; i < n; i++) {
    char ch;
    scanf(" %c", &ch);
```

```
    struct Node* newNode = create(ch);
```

```
    if (head == NULL) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }
}
```

```
scanf("%d", &index);
```

```
char val;
scanf(" %c", &val);
```

```
// check index
if (index >= n) {
    printf("Invalid index\n");
    printf("Updated list: ");
    printList(head);
    return 0;
}
```

```
// traverse to index
struct Node* temp = head;
for (int i = 0; i < index; i++) {
    temp = temp->next;
}
```

```
// insert after index
struct Node* newNode = create(val);
newNode->next = temp->next;
```

```
temp->next = newNode;  
printf("Updated list: ");  
printList(head);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Imagine Anu is tasked with finding the middle element of a doubly linked list. Given a doubly linked list where each node contains an integer value and is inserted at the end, implement a program to find the middle element of the list. If the number of nodes is even, return the middle element pair.

Input Format

The first line of input consists of an integer N, representing the number of nodes in the doubly linked list.

The second line consists of N space-separated integers, representing the values of the nodes in the doubly linked list.

Output Format

The first line of output prints the space-separated elements of the doubly linked list.

The second line prints the middle element(s) of the doubly linked list, depending on whether the number of nodes is odd or even.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 10 20 30 40 50

30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// create node
struct Node* create(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = newNode->next = NULL;
    return newNode;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node *head = NULL, *tail = NULL;

    // input
    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);

        struct Node* newNode = create(x);

        if (head == NULL) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}
```



```

// print list
struct Node* temp = head;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
printf("\n");

// find middle
int mid1, mid2;

if (n % 2 == 1) {
    temp = head;
    for (int i = 0; i < n / 2; i++) {
        temp = temp->next;
    }
    printf("%d", temp->data);
} else {
    temp = head;
    for (int i = 0; i < n / 2 - 1; i++) {
        temp = temp->next;
    }
    mid1 = temp->data;
    mid2 = temp->next->data;
    printf("%d %d", mid1, mid2);
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of positions to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

1 2 3 4 5

1

Output: 5 1 2 3 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* prev;  
    struct Node* next;  
};
```

```
// create node
```

```
struct Node* create(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->prev = newNode->next = NULL;  
    return newNode;
```

```
}
```

```
int main() {
```

```
    int n, k;
```

```
    scanf("%d", &n);
```

```
    struct Node *head = NULL, *tail = NULL;
```

```
    // input
```

```
    for (int i = 0; i < n; i++) {
```

```
        int x;
```

```
        scanf("%d", &x);
```

```
        struct Node* newNode = create(x);
```

```
        if (head == NULL) {
```

```
            head = tail = newNode;
```

```
        } else {
```

```
            tail->next = newNode;
```

```
            newNode->prev = tail;
```

```
            tail = newNode;
```

```
        }
```

```
    }
```

```
    scanf("%d", &k);
```

```
    // make circular
```

```
    tail->next = head;
```

```
    head->prev = tail;
```

```
    // find new head
```

```
    int steps = n - k;
```

```
    struct Node* newHead = head;
```

```
    for (int i = 0; i < steps; i++) {
```

```
        newHead = newHead->next;
```

```
    }
```

```
    struct Node* newTail = newHead->prev;
```

```
    // break circular
```

```
    newTail->next = NULL;
```

```
newHead->prev = NULL;
// print result
struct Node* temp = newHead;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}

return 0;
}
```

Status : Correct

Marks : 10/10